KasSigner

Security Architecture

An air-gapped signing device for Kaspa. This document describes what protects the keys, what does not, and where the limits are.

Release v1.0.6

Date August 2026

Licence GPL-3.0

Repository github.com/InKasWeRust/KasSigner

Contact kassigner@proton.me

Don't trust, verify. Sign offline. Broadcast sovereign.

1 · Trust Model

KasSigner is the trust anchor. KasSee is not.

KasSigner is air-gapped. No network stack, no WiFi, no Bluetooth, no USB data. Private keys exist only in volatile RAM and are destroyed when the device powers off. The device screen is the only trusted display in the system.

KasSee is a browser-based watch-only wallet running in an environment the user does not control. It never sees private keys. It builds unsigned transactions and broadcasts signed ones. A compromised browser, a phishing clone or a rogue extension can change what it shows. Every amount and address must be checked on the device before signing.

This is the whole model in one line: the browser proposes, the device discloses, the user decides.

What this document is honest about

Not every layer here is equally strong, and saying otherwise would be useless to a reader trying to judge risk. Where a layer is obfuscation rather than cryptography, it is labelled as such. Where a claim was disproved by testing, that is recorded rather than quietly dropped.

2 · Device Security Layers

1 **Air-gap enforcement**
The WiFi and Bluetooth radios are never initialised, and at boot the modem clocks are gated and the wireless power domain is switched off. USB OTG is disabled. Data moves only through QR codes, the SD card and the touchscreen. In production builds the USB Serial/JTAG peripheral is also gated shortly after boot, so a released device stops presenting a serial endpoint at all.

2 **Volatile key storage**
All key material lives in SRAM. Mnemonic, master key, derived keys and signing nonces are wiped after use and lost on power-off. Nothing is written to flash. The panic handler wipes RAM even on a crash.

3 **ROM Secure Boot v2**
On a provisioned device the ESP32-S3 ROM verifies an RSA-3072 signature against a digest burned permanently into eFuse, before any code runs. Only firmware signed with the matching private key executes. This is the layer that anchors everything above it.

4 **Software firmware verification**
The firmware hashes its own code segment at boot and checks a Schnorr signature over that hash. In a production build a mismatch halts. On its own this cannot stop an attacker who replaced the firmware and removed the check, since the hash, the signature and the public key all live inside the image being verified. ROM Secure Boot is what makes it meaningful.

5 **Memory safety**
Pure Rust, no_std. The signing path carries no unsafe code beyond volatile writes used to wipe secrets. Malicious input produces a panic and a RAM wipe, never arbitrary code execution.

6 **Encrypted backups**
SD card and steganographic backups share one container: AES-256-GCM with a per-file random salt and nonce, keyed through PBKDF2. Seeds, keys, multisig descriptors and unsigned transactions all use it, with the purpose bound into the key so one kind of container cannot be passed off as another.

7 **Steganographic hiding**
The encrypted seed hides inside an ordinary JPEG photograph. Two carriers, EXIF metadata and the image's own compressed data, which fail on opposite operations, and the photo's caption doubles as the encryption password. Which file matters is not visible; a trained detector remains a real adversary, and the design says so.

8 **BIP39 passphrase**
The optional 25th word derives a completely separate wallet. An attacker who recovers the 24 words, from a card, a backup or a photo, reaches only the decoy wallet; the real one needs the passphrase as well, and the passphrase is never stored on the device or in any backup it writes.

Build and boot integrity

Every released image is a reproducible Docker build; anyone can rebuild it and compare hashes. And on every power-on, before anything else is usable, the firmware checks its cryptographic primitives against published answers and refuses to boot if any of them disagree: BIP39, BIP32, Schnorr, the 45' multisig address against an independent implementation, and 27 transaction sighash vectors taken from the rusty-kaspa 2.0.1 consensus tests across all six sighash types and both transaction versions. A device whose sighash diverges from the node by one field does not warn; it does not boot.

3 · eFuse Hardening

eFuses are permanent. Burning them is one-way, and a mistake bricks the board. The configuration below is what a provisioned KasSigner carries.

eFuse	State	Effect
SECURE_BOOT_EN	burned	ROM rejects unsigned firmware
KEY0 / KEY1 digests	burned, write-protected	RSA-3072 public key digests
DIS_PAD_JTAG	burned	Hardware JTAG closed
DIS_USB_JTAG	burned	USB-to-JTAG bridge closed
DIS_USB_SERIAL_JTAG	left unburned	Serial console and flashing preserved
DIS_DOWNLOAD_MODE	left unburned	Signed updates remain possible

Secure Boot does not close download mode

Verified on a provisioned board in August 2026: with Secure Boot and both JTAG fuses burned, every download-mode fuse still read false and the board flashed normally. That is correct, not a gap. Flashing has to stay open so signed updates remain possible; what Secure Boot enforces is that only firmware signed with the burned digest will run. An attacker can write whatever they like and the ROM refuses to execute it.

Two fuses must not be burned. DIS_USB_SERIAL_JTAG disables the whole peripheral, permanently removing the console and the only path for signed updates. DIS_DOWNLOAD_MODE removes the recovery path entirely, which matters twice over on a production build that already gates USB after boot.

4 · The KasSee Boundary

KasSee is a convenience tool, not a security boundary.

It imports the extended public key, derives watch-only addresses, fetches balances, builds unsigned transactions, and broadcasts signed ones. It never touches private keys. But it runs in a browser, on an operating system, over a network, and any of those can be compromised.

A phishing clone can show one address and encode another in the QR. Browser malware can rewrite a transaction in memory. The WebAssembly is built from the same open source and can be checked against a reproducible build, which raises the bar, but it does not replace the final check: verify on the device screen. The device shows what is actually in the transaction data, not what the browser claims.

This extends to covenants. The device does not decode covenant semantics; it shows the destination and amount of every output, the payload hash and the covenant id when present, and first checks that the redeem script the host supplied hashes to the P2SH commitment being spent, so a substituted script cannot vouch for itself. The trust anchor for a covenant is the covenant address you verified out of band when it was created, matched against what the device displays.

For 45' multisig, the descriptor is a second secret: a seed alone cannot find or spend the funds, so back up seed and descriptor. When a transaction claims an output as change, the device tries to reproduce it from a descriptor that already reproduces one of the inputs; if it cannot, the claim is shown as unverified, and if the descriptor contradicts it, the device refuses to sign.

Privacy

By default KasSee talks to a public Kaspa node, which can see which addresses belong to one wallet, the balance, and the user's IP. Running your own node removes that. Stealth payments add address-level privacy on chain, with a view key for scanning that is separate from the spend key.

5 · Attack Surface

Powered-off device
Nothing to extract. SRAM is gone and flash holds only open-source firmware. An attacker who also takes the SD card faces AES-256-GCM and then the passphrase.

Powered-on device, seed loaded
The only genuinely vulnerable state. Voltage glitching, cold boot, EM side-channel and power analysis all require physical possession and lab equipment. The mitigation is behavioural: load, sign, power off. The device helps: it wipes all loaded seeds after four minutes idle, and has a hold-to-confirm manual wipe.

Firmware tampering
On a provisioned device the ROM rejects anything unsigned, and the attacker needs an offline RSA key. Without eFuses the software check warns on a dev build and halts on a production build, but whoever holds the device can remove it; Secure Boot is what makes it stick.

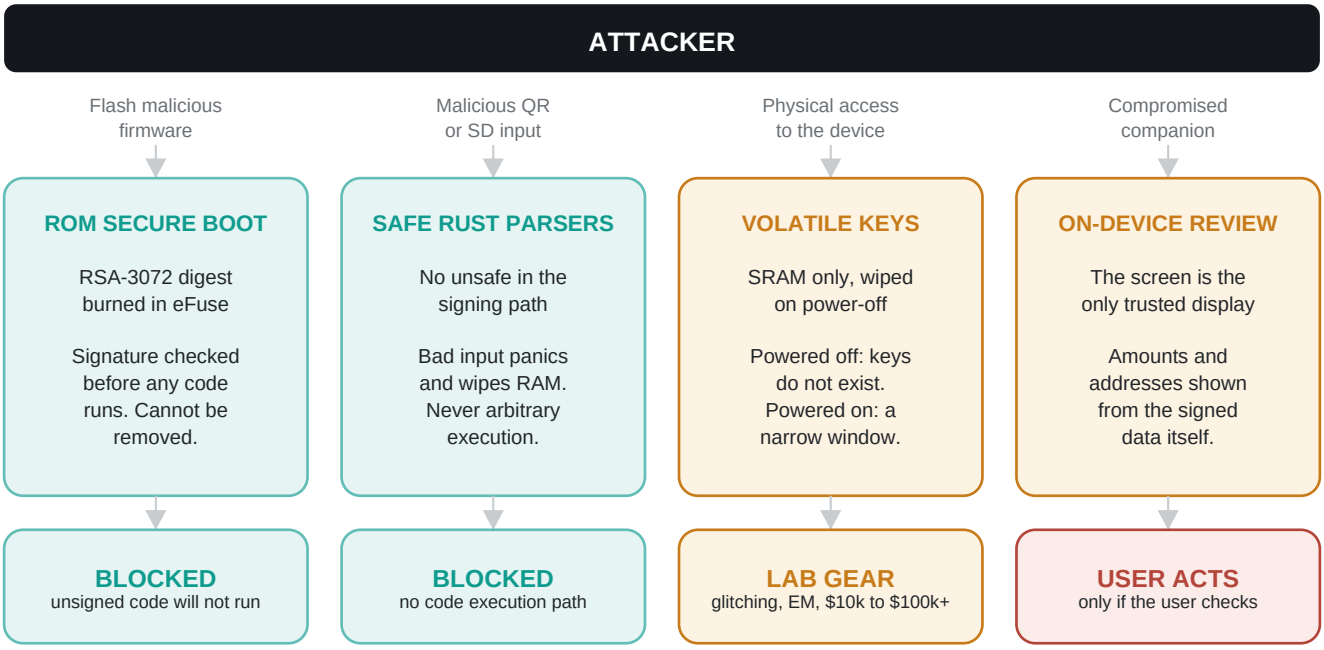
Malicious QR or SD input
Every parser is safe Rust. Hostile data produces a panic and a RAM wipe rather than code execution.

Compromised companion
The most likely real-world attack, and the one the technology cannot close alone. It is stopped by the user reading the device screen.

Stolen backup
An SD card or a stego photo bypasses the device entirely. Encryption buys time; the passphrase is what actually holds.

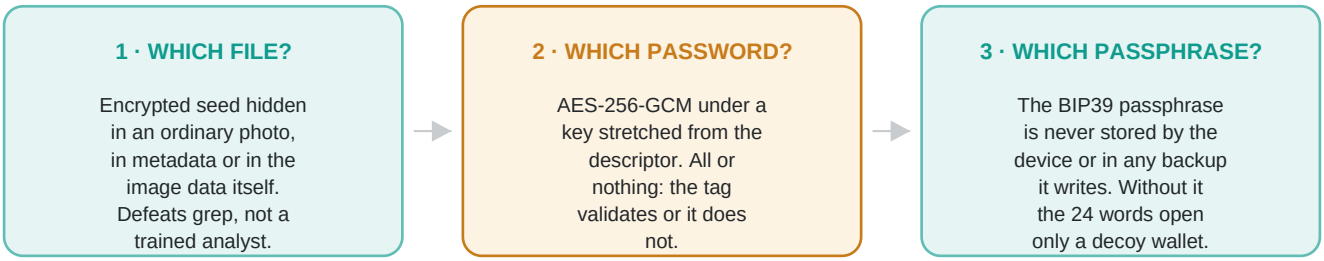
6 · Attack Barriers

Five ways in. What each one runs into, and what it costs to get past it.



A stolen backup takes a different route

An SD card or a stego photo does not need the device at all. Three walls, and only the last one is load-bearing.



Cryptographic or silicon barrier · Real cost, not impossible · Depends on the user

7 · Where It Sits

Against a hardware wallet with a secure element, and against a software wallet. KasSigner is not strictly better than either; it trades differently.

	KasSigner	Ledger / Trezor	Software wallet
Key storage	Volatile RAM only	Secure element	Disk or memory
Secure element	No	Yes	No
Air-gapped	Yes, completely	No, USB or BT	No
Firmware auditable	100% open Rust	Partially	Varies
Physical attack	Limited defence	Strong defence	None
Keys after power-off	Do not exist	Held in the SE	Encrypted at rest
Network surface	Zero	USB or BT stack	Full OS stack
Companion trust	None, device verifies	Partial	Full

The honest summary: no secure element means less resistance to a well-equipped attacker holding a powered device. Complete air-gapping and zero persistent key storage means there is far less to attack in the first place.

8 · Reproducible Build

Anyone can check that a released binary was built from the published source. Every input is pinned: the base image by content digest, the system packages by version, the host and cross compilers by version, and the dependencies by lockfile.

1. Clone the repository at the release tag.
2. Build the pinned toolchain image, then the firmware.
3. Compare against the published unsigned hashes.

Signed and unsigned images differ, because the signature is embedded in the firmware. A verifier has no signing key, so their build is unsigned and must be compared against the unsigned figures. The device also displays a hash of its own code segment at boot, which on a production build is proof that the running code matches what was verified.

Full procedure in docs/REPRODUCIBLE_BUILD.md.

9 · What Is Left To You

Three things the design cannot do on your behalf.

1 Read the screen before you sign.
The single most important habit in the whole system. No amount of engineering protects someone who approves a transaction without looking at it.

2 Protect the backup, and the passphrase separately.
The 24 words plus the passphrase are the wallet. Keep them apart. The device never stores the passphrase; where you keep it is yours to decide, and it should not be the same place as the words.

3 Power off when you are done.
The device is only attackable while it is on with keys loaded. That window is under your control, and it should be short.

KasSigner is experimental software on consumer hardware with no secure element. It has been through several security reviews, but not a paid engagement with an independent firm. Do not use it to hold more than you can afford to lose.